BillieBot Sensor Bench Test Plan

Document purpose: Define minimally coupled bench tests for four BillieBot sensors before vehicle integration.

Sensors covered:

1. Luxonis OAK-D Lite stereo camera
2. MLX90640 32 × 24 thermal camera
3. Raspberry Pi Camera Module 3 NoIR
4. Seeed Studio reSpeaker XVF3800 USB microphone array

Test classes:

- **Type 1 — Sensor acquisition/unit tests:** Verify that the host detects the hardware, the sensor produces valid data, ROS 2 topics publish at the intended rate, the data can be visualized, and recorded data can be exported and quantitatively analyzed.
- **Type 2 — Detector/classifier tests:** Route live sensor data through the corresponding BillieBot detector or classifier and measure detection behavior. The Pi Camera 3 NoIR has no Type 2 test in the current BillieBot architecture.

Source baseline:

- Repository: `sean-mackenzie/billie-bot-claude`
- Repository baseline reviewed: `main` at commit `1df1f792598b7f178ae57d58a4359c508f22e926` (2026-08-01)
- Primary source document: `BRINGUP_LADDER_ANALYSIS.md`
- Relevant production nodes and configuration:
  - `billiebot_perception/oakd_dog_detector.py`
  - `billiebot_perception/thermal_node.py`
  - `billiebot_perception/noir_cam_node.py`
  - `billiebot_audio/audio_classifier.py`
  - `billiebot_perception/config/perception.yaml`
  - `billiebot_audio/config/audio.yaml`
  - bringup rungs `07_oakd`, `09_thermal`, `10_noir`, and `11_audio`

Implementation status (as of `19f8513`, merged 2026-08-01): the `billiebot_sensor_tests` package described in §3.1, the four production changes in §5 of the implementation record, and the associated tests all landed in PR #32 (commit `97c9339`). This document has been updated to match the as-built package (correct package name, actual module layout, corrected default values) throughout. Wherever the shipped implementation is narrower than what this spec originally called for, an **Implementation note** callout marks the gap in place — those are genuine follow-up items, not spec changes. §11 tracks the overall resolved/partial/open status per finding.

---

1. Scope and Test Philosophy

These tests intentionally exclude the BillieBot chassis, drive train, lidar, navigation stack, map, mission controller, and multi-computer DDS configuration unless one of those elements is strictly necessary for sensor verification. Each sensor is tested by itself on the computer intended to host it in the deployed system:

Sensor	Preferred bench host	Physical interface	Production role
OAK-D Lite	Jetson Orin Nano	USB 3.x	RGB dog detection and stereo range
MLX90640	Raspberry Pi 5	I²C bus 1, address `0x33`	Thermal image and warm-body blob
Pi Camera 3 NoIR	Raspberry Pi 5	CSI-2 camera connector	Low-light/near-IR imagery
reSpeaker XVF3800	Raspberry Pi 5	USB	Audio capture, YAMNet classification, optional DoA

A MacBook running Foxglove Studio is used only as a visualization workstation. It does not need to participate in ROS 2 discovery. Each bench launch should start `foxglove_bridge` on the sensor host, and Foxglove Studio should connect over WebSocket.

1.1 Important corrections to the initial draft

1. **The Pi Camera 3 NoIR is not a thermal camera.** It cannot measure an object at 35 °C against a 22 °C background. Its quantitative test must measure visible/near-IR image quality, focus, sensitivity, contrast, noise, and low-light performance.
2. **Event topics do not necessarily publish at the processing-loop rate.** In particular, `/audio/events` publishes only when energy and confidence gates are passed. A separate diagnostic/heartbeat topic is needed to verify classifier processing frequency.
3. **The current audio implementation cannot realistically execute a 2 Hz classification loop while blocking for a 0.975 s recording inside each 0.5 s timer callback.** The bench test should measure this and the implementation should be refactored to use a continuous stream or ring buffer with a 0.5 s hop if 2 Hz processing is required.
4. **Scientific data should be stored in a lossless numeric format.** PNG/JPEG images are useful for inspection, but depth and temperature data should also be saved as NumPy `.npz`/`.npy`, float TIFF, PLY/PCD, or rosbag2 data so physical units are preserved.
5. **Detector tests should exercise the production detector logic.** A duplicate test-only detector can give false confidence. Test-only publishers, visualization, diagnostics, and recorders may be added, but the algorithm under test should be the production node.

---

2. Common Bench Equipment and Safety

2.1 Common equipment

- Jetson Orin Nano with the BillieBot ROS 2 workspace built and sourced
- Raspberry Pi 5 with the BillieBot ROS 2 workspace built and sourced
- MacBook with Foxglove Studio
- Sensor-appropriate data cables
- Stable sensor mount or small tripod
- Tape measure or laser distance meter
- Printed checkerboard or high-contrast test chart
- Large flat, rigid, textured board for stereo depth testing
- Matte black electrical tape or another high-emissivity target surface
- Contact thermometer or thermocouple for thermal reference measurements
- Controlled-temperature target, such as a water-filled container with a matte black measurement patch
- Optional 850 nm IR illuminator for the NoIR test
- External powered speaker for repeatable audio playback
- Quiet room or a room with documented ambient noise

2.2 Safety and handling

- Power down the Raspberry Pi before connecting or disconnecting the CSI ribbon cable.
- Power down the Raspberry Pi before changing GPIO/I²C wiring.
- Use ESD precautions when handling camera boards and exposed sensor PCBs.
- Confirm the MLX90640 breakout board's allowed supply voltage before wiring. The test plan uses 3.3 V because it is safe for the Raspberry Pi I/O domain, but the exact breakout documentation governs.
- Do not force Billie to bark or remain in an uncomfortable test position. Use prerecorded Billie bark samples for repeatable classifier characterization and use live barks only as a final confirmation.
- Keep hot reference objects at safe temperatures and away from Billie.

---

3. Common Software Test Architecture

3.1 Recommended package layout

Claude Code should implement the bench suite under a new `billiebot_sensor_tests` package with the following structure:

```
billiebot_ws/src/billiebot_sensor_tests/
├── package.xml, setup.py, setup.cfg, resource/billiebot_sensor_tests, README.md
├── config/
│   └── sensor_bench.yaml
├── foxglove/
│   └── billiebot_sensor_bench.json
├── launch/
│   ├── oakd_unit_bench.launch.py
│   ├── oakd_detection_bench.launch.py
│   ├── thermal_unit_bench.launch.py
│   ├── thermal_detection_bench.launch.py
│   ├── noir_unit_bench.launch.py
│   ├── audio_capture_bench.launch.py
│   └── audio_classifier_bench.launch.py
├── billiebot_sensor_tests/
│   ├── common/            # shared infra: result_dir, manifest, config, rate_stats,
│   │                       # rate_monitor, image_hash, circular_stats, bag_reader,
│   │                       # launch_helpers, preflight, ground_truth_marker, report
│   ├── orchestrate/        # test_registry, run_sensor_test, finalize_check
│   ├── oakd/               # oakd_bench_publisher, oakd_preview_overlay, metrics,
│   │                       # analyze_oakd_depth, detection_scoring, score_oakd_detector
│   ├── thermal/            # thermal_colorizer, metrics, analyze_thermal_frame,
│   │                       # blob_scoring, score_thermal_blob
│   ├── noir/               # image_quality_monitor, metrics, analyze_noir_image
│   └── audio/              # audio_capture_node, metrics, analyze_audio,
│                           # classifier_scoring, score_audio_classifier
└── test/
    ├── fixtures/    # common_, oakd_, thermal_, noir_, audio_fixtures.py
    ├── test_common_metrics.py
    ├── test_oakd_metrics.py
    ├── test_thermal_metrics.py
    ├── test_noir_metrics.py
    ├── test_audio_metrics.py
    └── test_launch_files.py
```

> **Implementation note:** the shipped layout co-locates the CLI (`analyze_*`/`score_*.py`) with the pure-metric modules and the ROS acquisition nodes inside each sensor's own subpackage, rather than using separate top-level `analysis/`/`scripts/` directories — hardware imports stay lazy and scoped to acquisition-node files only, so this still satisfies "analysis code never imports hardware libraries." Every script is an installed `console_scripts` entry point, not a loose file under `scripts/`. `topic_rate_monitor.py`/`test_manifest_node.py` became `common/rate_monitor.py` (class `TopicRateMonitorNode`) and `common/manifest.py` (class `ManifestWriter`, invoked from a launch-time `OpaqueFunction` rather than a standalone

> node); `classifier_diagnostics.py`'s responsibility is met by `/bench/audio_classifier/status`, published directly by the production `audio_classifier` node (§8.2) rather than a separate bench node.

3.2 Common launch arguments

Every bench launch should support these arguments where applicable:

Argument	Example	Purpose
`results_dir`	`~/billiebot_test_results/UT-OAK-01_20260801T130000`	Root output directory
`duration_sec`	`60`	Automated recording duration
`record_bag`	`true`	Start rosbag2 recording
`start_foxglove`	`true`	Start `foxglove_bridge`
`foxglove_port`	`8765`	WebSocket port
`config_file`	package config path	Test parameters and acceptance limits
`sensor_serial`	optional	Select a specific USB camera if more than one is connected
`fail_on_missing_device`	`true`	Exit nonzero instead of idling

3.3 Common test output format

Each test execution should create:

```
<results_dir>/
├── manifest.yaml
├── console.log
├── bag/
├── exports/
├── plots/
├── metrics.json
├── metrics.csv
└── report.md
```

`manifest.yaml` should contain at least:

```yaml
test_id: UT-OAK-01
start_time_utc: 2026-08-01T18:00:00Z
host: billiebot-jetson
host_os: Ubuntu
ros_distro: <detected>
repository_commit: <git SHA>
sensor_model: OAK-D Lite
sensor_serial: <detected or operator-entered>
launch_command: <full command>
parameters: {}
ground_truth: {}
operator_notes: ""
```

`metrics.json` should provide machine-readable pass/fail results. Every analysis script should exit `0` on pass and nonzero on failure.

> **Implementation note:** `manifest.yaml`'s `parameters:` field is currently always written as `{}` — the effective ROS parameters used by each launch are not yet captured into the manifest, despite §10.1/§10.4 requiring this. This is an open gap, not a design choice; closing it means threading the resolved launch-argument values (or a `ros2 param dump`-equivalent) into `common/manifest.py`'s `write_initial()` call. The shipped manifest does capture more than this minimal example otherwise: `repository_dirty`, `config_file`, `end_time_utc`, `exit_code`, `software_versions`, and `detected_hardware_interfaces` keys are also present.

3.4 Foxglove connection procedure

1. Start the applicable bench launch on the Jetson or Raspberry Pi.
2. Determine the host IP address with `hostname -I`.
3. On the MacBook, open Foxglove Studio.
4. Add an **Open connection** connection using:

   ```
   ws://<sensor-host-IP>:8765
   ```

5. Add the panels specified by the individual test.
6. Save a Foxglove layout named `BillieBot Sensor Bench` so it can be reused.

> **Implementation note:** step 6 is already done for you — import the shipped layout instead of building one by hand: `foxglove/billiebot_sensor_bench.json`, installed to `share/billiebot_sensor_tests/foxglove/`. It is one shared layout (a Tab panel with an OAK-D/Thermal/NoIR/Audio tab plus a shared `/bench/status` + Log strip) covering every test in this document, not a per-test layout.

Foxglove is a qualitative visualization tool in this plan. Numeric acceptance is determined by recorded data and analysis scripts.

3.5 Common rate and integrity monitor

Implement one reusable `topic_rate_monitor.py` node that:

- subscribes to one or more configured topics;
- records message count, first/last timestamp, mean frequency, median frequency, minimum/maximum period, period standard deviation, and maximum observed gap;
- checks message header timestamps when available;
- checks monotonic timestamps;
- optionally computes repeated-frame hashes for image streams;
- writes periodic diagnostics and a final JSON file;
- publishes a test status topic such as `/bench/status` at 1 Hz;
- exits nonzero if a required topic never appears or violates a configured threshold.

> **Implementation note:** shipped as `common/rate_monitor.py`'s `TopicRateMonitorNode`, configured per launch file via a JSON-encoded `topics_config_json` parameter (topic name, message type, required flag, optional `min_rate_hz`, optional `hash_enabled` for repeated-frame detection). Its exit code (when run standalone via `ros2 run`) reflects pass/fail, but the authoritative signal every launch actually relies on is the `exports/topic_rate_monitor.json` file it writes, read back by `orchestrate/finalize_check.py`.

---

4. Test Summary and Acceptance Matrix

Thresholds marked **provisional** are engineering gates for initial bench acceptance, not claims of sensor datasheet accuracy. They should remain configurable in `sensor_bench.yaml`.

Test ID	Test	Primary acceptance
UT-OAK-01	OAK-D stream and visualization	RGB/depth data valid; requested 5 Hz stream averages at least 4.5 Hz; no device resets
UT-OAK-02	OAK-D 2 m depth accuracy	Median depth error ≤ 0.20 m; valid ROI depth ≥ 90%; plane-fit residual target ≤ 0.05 m provisional
DT-OAK-01	OAK-D dog detection	`/dog/found` near 5 Hz; dog recall ≥ 85% through 4 m where scene geometry permits; 2 m depth error ≤ 0.20 m
UT-THM-01	MLX90640 stream	32 × 24 `32FC1`; mean rate 3.6–4.4 Hz; ≥99.9% finite pixels; no sustained read errors
UT-THM-02	MLX90640 temperature contrast	Target/background biases each ≤ 2 °C provisional; contrast-to-noise ratio ≥ 5 provisional
DT-THM-01	Thermal warm-body blob	No detections in empty baseline; detection in ≥80% of frames at 0.5–1.5 m provisional; area ≥8 px; temperature 30–40 °C
UT-NIR-01	NoIR image stream	640 × 480 `rgb8`; mean rate ≥4.5 Hz; repeated/dropped-frame fraction ≤1%
UT-NIR-02	NoIR image quality/low light	Autofocus stable; normal-light CNR ≥10 provisional; IR-assisted low-light CNR ≥5 provisional
UT-AUD-01	XVF3800 audio acquisition	Valid WAV, 16 kHz, expected channel count, 3.00 ±0.05 s, no clipping, audible playback
DT-AUD-01	Audio classification	Processing loop 1.8–2.2 Hz after refactor; bark recall ≥80%; false bark rate on speech/noise ≤10% provisional
DT-AUD-02	XVF3800 DoA, optional	Mean circular error ≤15° at cardinal bearings, consistent with system requirement

---

5. OAK-D Lite Tests

5.1 UT-OAK-01 — Basic RGB, Stereo Depth, and Point-Cloud Acquisition

Goal

Verify that the Jetson detects the OAK-D Lite as a USB device, DepthAI can open it, synchronized RGB and stereo depth data can be acquired, the data can be viewed in real time in Foxglove Studio, and the streams meet the intended approximately 5 Hz bench rate.

Hardware and hookup

- Jetson Orin Nano, powered normally
- OAK-D Lite
- Known-good USB-C data cable capable of USB 3.x
- Direct Jetson USB 3.x port; avoid an unpowered hub for the initial test
- Stable camera mount
- MacBook on the same IP network for Foxglove

Connect the OAK-D Lite to the Jetson by USB only. No robot wiring or external power is required.

Hardware preflight

The test runner should execute and save the output of:

```
lsusb
lsusb -t
python3 -c "import depthai as dai; print(dai.__version__); print(dai.Device.getAllAvailableDevices())"
```

Preflight acceptance:

- The OAK-D appears in the USB inventory.
- `lsusb -t` reports a SuperSpeed connection when possible rather than a 480 Mbit/s USB 2 fallback.
- DepthAI reports exactly one intended device and can open it without a permission or XLink error.

Required test-node behavior

The current production `oakd_dog_detector` publishes dog detections and `/dog/found`, but it does **not** publish RGB, depth, or point-cloud topics. Implement a dedicated acquisition node or a controlled test mode that publishes:

Topic	Type	Recommended content
`/bench/oakd/rgb/image_raw`	`sensor_msgs/Image`	RGB or BGR image, clearly documented encoding
`/bench/oakd/rgb/camera_info`	`sensor_msgs/CameraInfo`	Intrinsics from device calibration
`/bench/oakd/depth/image_raw`	`sensor_msgs/Image`	Registered depth, preferably `16UC1` in mm or `32FC1` in m
`/bench/oakd/depth/camera_info`	`sensor_msgs/CameraInfo`	Depth camera calibration
`/bench/oakd/points`	`sensor_msgs/PointCloud2`	XYZ or XYZRGB point cloud in the camera optical frame
`/bench/oakd/diagnostics`	`diagnostic_msgs/DiagnosticArray`	Device, USB speed, frame counts, dropped queues, temperatures if available

The acquisition node should use the device calibration rather than hard-coded intrinsics. It should expose resolution, FPS, stereo preset, left-right check, subpixel, extended disparity, and depth alignment as parameters.

> **Implementation note:** shipped as `oakd/oakd_bench_publisher.py` (class `OakdBenchPublisher`). It reads device calibration via `readCalibration()` and exposes `fps`, `lr_check`, `subpixel`, `extended_disparity`, `depth_align`, `sensor_serial`, and `point_cloud_stride` as parameters, matching the production node's stereo/color resolution and `HIGH_DENSITY` preset rather than exposing `resolution`/`stereo_preset` as separate configurable parameters — an open gap if independent resolution/preset sweeps are ever needed for this test.

Launch-file outline

`oakd_unit_bench.launch.py` should:

1. Launch the OAK-D acquisition node in real mode.
2. Launch `topic_rate_monitor.py` for the RGB, depth, and point-cloud topics.
3. Launch a rosbag2 recorder for all OAK-D bench topics.
4. Launch `foxglove_bridge` when `start_foxglove:=true`.
5. Launch a timed shutdown or result-finalization node when `duration_sec` is nonzero.
6. Fail the launch if the device cannot be opened.

Example intended command:

```
ros2 launch billiebot_sensor_tests oakd_unit_bench.launch.py \
  results_dir:=$HOME/billiebot_test_results/UT-OAK-01 \
  duration_sec:=60 \
  record_bag:=true \
  start_foxglove:=true
```

Procedure

1. Place the camera on a stable mount facing a room with objects at multiple depths.
2. Run the hardware preflight.
3. Start the launch file.

4. In Foxglove:
    - add an **Image** panel for `/bench/oakd/rgb/image_raw`;
    - add an **Image** panel for `/bench/oakd/depth/image_raw` and select an appropriate depth colormap/range;
    - add a **3D** panel for `/bench/oakd/points`;
    - add a **Raw Messages** or **Plot** panel for `/bench/oakd/diagnostics`.
5. Move a textured object toward and away from the camera and confirm that the depth image and point cloud change coherently.
6. Leave the sensor stationary for the remainder of the 60 s recording.
7. Stop the launch cleanly and verify that all result files are finalized.

Analysis outline

The automated analysis should check:

- topic type, image dimensions, encoding, frame ID, and timestamp monotonicity;
- average/median rate and maximum inter-frame gap;
- RGB/depth timestamp offset for synchronized frames;
- fraction of invalid depth pixels (`0`, NaN, Inf, or outside configured depth limits);
- point-cloud width/height and finite XYZ fraction;
- device or queue errors in `console.log`.

> **Implementation note:** `oakd/analyze_oakd_depth.py --profile stream` checks topic presence, dimensions/encoding, and rate/gap/monotonicity (via `common/rate_stats.py`, using header timestamps). It does not yet compute the RGB/depth timestamp offset, the invalid-depth-pixel fraction, or the point-cloud finite-XYZ fraction as part of this profile (the invalid-pixel-fraction math itself exists in `oakd/metrics.py`'s `valid_pixel_fraction()` and is exercised by the `--profile accuracy` path for UT-OAK-02, just not wired into the stream profile here), and it does not scan `console.log` for device/queue errors. These are open gaps for a more complete UT-OAK-01 analysis pass.

Pass/fail criteria

Pass if all of the following are true:

- RGB, depth, camera-info, and point-cloud topics are present.
- The requested 5 Hz output averages at least 4.5 Hz over a 60 s run.
- No inter-frame gap exceeds 1.0 s.
- Header timestamps are monotonic.
- The scene is recognizable in RGB and geometrically coherent in depth/point cloud.
- The device does not reset or disconnect.

A USB 2 connection is not automatically a failure at 5 Hz, but it should be reported as a warning because it can limit later high-rate operation.

> **Implementation note:** the first four criteria above are programmatically checked; "scene recognizable"/"geometrically coherent" remain qualitative Foxglove review as the spec intends. Device-reset/disconnect detection and the USB-2-vs-USB-3 warning are not automated — `common/preflight.py` captures `lsusb -t` output for manual review, but no analysis code parses it or gates on it yet (this is a general pattern across every sensor's preflight, not specific to OAK-D — see the cross-cutting note in §10.1).

5.2 UT-OAK-02 — Flat-Target Depth Accuracy and Precision

Goal

Measure range bias, repeatability, valid-pixel fraction, and planar noise for a target at a known distance, with 2.0 m as the required acceptance point.

Hardware and target setup

- Equipment from UT-OAK-01
- Large flat, rigid board, preferably at least 0.6 m × 0.6 m
- A printed random-dot pattern, newspaper, or other texture attached flat to the board; a blank low-texture wall is a poor stereo target
- Tape measure or laser distance meter
- Bubble level or careful perpendicular alignment

Place the board approximately perpendicular to the camera optical axis. Measure the reference distance from a consistently defined camera reference plane to the target plane. Document the exact reference point in `manifest.yaml`.

Required distance:

- 2.00 m

Recommended characterization distances:

- 1.00 m
- 2.00 m
- 3.00 m
- 4.00 m, if the room permits

Launch and data capture

Use the UT-OAK-01 launch with a `test_mode:=flat_target` argument. Record at least 10 s of stationary data at each distance. Save:

- rosbag2 data;
- one representative RGB PNG;
- one depth frame as float `.npz` and optionally 32-bit TIFF;
- one point cloud as PLY or PCD;
- a JSON file containing the measured ground-truth distance and target ROI.

The authoritative numeric record should be the rosbag2 data or `.npz`, not a colorized image.

> **Implementation note:** rosbag2 is the authoritative record, as specified — but `analyze_oakd_depth.py` does not currently export a representative RGB PNG, depth `.npz`, or point cloud to `exports/` on its own; the bag itself remains the only artifact written. This is an open gap, shared across UT-OAK-01/02, UT-THM-01/02, and UT-NIR-01/02 (see the cross-cutting note in §10.1) — an operator can extract representative frames from the bag manually in the meantime.

Analysis-script outline

`analyze_oakd_depth.py` should:

1. Load synchronized RGB and depth frames from rosbag2 or exported arrays.
2. Load the configured target ROI or allow the operator to select a polygon on the representative RGB image.
3. Map the ROI to aligned depth coordinates.
4. Reject invalid depth values and configured outliers.
5. Compute per frame:
   - valid-pixel fraction;
   - median range;
   - mean range;
   - bias = median range − ground truth;
   - standard deviation;
   - robust standard deviation = `1.4826 × MAD`;
   - 5th, 50th, and 95th percentiles;
   - fraction of points outside ±0.20 m of ground truth.
6. Convert ROI pixels to XYZ using camera intrinsics, or load the point cloud.
7. Fit a plane using SVD or RANSAC.
8. Compute:
   - point-to-plane RMSE;
   - fitted plane normal;
   - angle between fitted normal and the expected optical-axis direction;
   - plane distance from the camera.
9. Aggregate metrics across all frames.
10. Generate a histogram, depth heatmap, plane-residual plot, and Markdown report.

Plane fitting is preferred over only taking the standard deviation of `z`, because even a small target tilt produces a real depth gradient that is not sensor noise.

> **Implementation note:** implemented in `oakd/metrics.py` (`fit_plane_svd` via `numpy.linalg.svd`, `point_to_plane_rmse`, `plane_normal_angle_deg`, `robust_std_mad` = 1.4826×MAD, `percentiles`, `fraction_within_band`) and `oakd/analyze_oakd_depth.py --profile accuracy`. The target ROI is a CLI-supplied axis-aligned rectangle (`--roi <path-to-json>` with `x0,y0,x1,y1`), not an interactive polygon-selection tool, and the selected ROI is only persisted if the operator saves that JSON file themselves — there is no automatic "save the selected ROI" step. Plane distance from the camera is not separately reported (only the fitted centroid/normal/RMSE/angle).

Pass/fail criteria at 2.0 m

- Absolute median depth bias ≤ 0.20 m.
- Valid depth fraction in the selected target ROI ≥ 90%.
- At least 95% of valid ROI pixels fall within ±0.20 m of the ground truth.
- Point-to-plane RMSE ≤ 0.05 m **provisional**.
- No device errors or frame discontinuities occurred during the capture.

The BillieBot requirement is the ±0.20 m acceptance gate. A stretch goal of approximately ±0.05 m at 2 m is reasonable for a well-textured, well-aligned indoor target and should be reported separately rather than replacing the system requirement.

5.3 DT-OAK-01 — Live Dog Detection and Stereo Range

Goal

Verify that the production `oakd_dog_detector` detects Billie when present, does not report a dog in controlled negative scenes, publishes at the intended cadence, and provides usable stereo range. Characterize the minimum and maximum reliable detection distances.

Hardware and hookup

Use the UT-OAK-01 setup. The chassis and robot TF tree are not required. `dog_locator` and `/dog/pose_map` are intentionally excluded because this test is limited to camera-frame detection and depth.

Production-software preconditions

- `depthai` is installed.

- The YOLO blob exists at the configured path, currently `/home/sean/billiebot/models/yolov8n_416.blob` in `perception.yaml`.
- The launch is run with `mock:=false`.
- The production node exits nonzero if the model or device is missing.

Known implementation item that the test should expose

DepthAI detection coordinates are normalized. The current production code converts `xmin`, `ymin`, and the width/height differences directly to integers, which can produce zero-valued bounding boxes. Before final acceptance, Codex should scale these values by the 416 × 416 preview dimensions or publish normalized coordinates in a message explicitly designed for them. The test scorer should fail if a positive detection has a nonpositive bounding-box width or height.

> **Implementation note — resolved:** fixed in `oakd_dog_detector.py` via a new module-level pure function `scale_bbox(xmin, ymin, xmax, ymax, frame_w=416, frame_h=416)` (rounds + clamps the normalized coordinates to pixel space), called unconditionally from `real_detect()` — this is a correctness fix with no parameter gating it, per Appendix B-11. `score_oakd_detector.py`'s `valid_bbox_fraction` check is the regression test proving it stays fixed.

Recommended test-only additions to the production node

Add optional parameters that default off in deployment:

```
publish_preview: false          # bench detection-bench launch overrides to true
publish_depth_preview: false    # bench detection-bench launch overrides to true
publish_diagnostics: false      # bench detection-bench launch overrides to true
```

Recommended topics:

- `/oak/rgb/preview` — published by `oakd_dog_detector` when `publish_preview:=true`
- `/oak/rgb/annotated` — **published by the bench package**, not the production node (see below)
- `/oak/depth/preview` — published by `oakd_dog_detector` when `publish_depth_preview:=true`
- `/dog/detections_3d`
- `/dog/found`
- `/bench/oakd_detector/diagnostics` — published by `oakd_dog_detector` when `publish_diagnostics:=true`

The preview should come from the same DepthAI pipeline and frame sequence that feeds the spatial detector.

> **Implementation note:** the shipped defaults are `false` for all three params (matching "default off in deployment" literally — the original draft's `true` defaults above were an error in the initial spec). There is no `publish_annotated_preview` parameter on the production node at all: annotation is done entirely bench-side by `oakd/oakd_preview_overlay.py`, which subscribes to `/oak/rgb/preview` + `/dog/detections_3d` and draws the bbox outline in pure numpy (no cv2 dependency), publishing `/oak/rgb/annotated` itself. This keeps the production node's only new behavior at "stream two more topics," not "run drawing code."

Launch-file outline

`oakd_detection_bench.launch.py` should:

1. Include or replicate the behavior of `07_oakd.launch.py` with `mock:=false` and the production configuration.
2. Enable test-only preview/diagnostic outputs.
3. Launch a recorder for all relevant topics.
4. Launch a scorer-assist node that records `/dog/found`, detections, confidence, bounding boxes, and depth with timestamps.
5. Launch Foxglove bridge.
6. Accept a ground-truth segment file or provide a service/keyboard interface for the operator to mark `dog_present`, distance, orientation, and test condition.

> **Implementation note:** step 4's "scorer-assist node" is the rosbag2 recording itself (item 3) plus the offline `oakd/score_oakd_detector.py` CLI, run after the session ends — there is no separate live node computing scores during the recording; this matches the plan's own principle that pass/fail is always derived from recorded data, not a live process. Step 6 is implemented as a stdin-driven interface: `common/ground_truth_marker.py`'s `ground_truth_marker_node` reads lines typed by the operator in the form `mark <label> [key=value ...]` (e.g. `mark dog_present distance=1.5m orientation=front condition=normal_light`), timestamps each with the ROS clock, and appends a row to `exports/ground_truth_segments.csv` — not a new ROS service.

Test scenes

Record at least 10 s per condition after Billie is settled:

Positive distances:

- 0.75 m, characterization only because stereo performance can degrade at very short range
- 1.0 m
- 1.5 m
- 2.0 m
- 3.0 m
- 4.0 m
- farther distances if the room permits

At selected distances, repeat with:

- Billie facing the camera;
- Billie side-on;
- Billie curled or lying down;
- normal indoor lighting and moderately dim lighting.

Negative scenes:

- empty scene for at least 30 s;
- a human in frame;
- a blanket, pillow, or stuffed animal;
- room motion without Billie.

Foxglove procedure

- View `/oak/rgb/annotated` in an Image panel.
- View `/dog/detections_3d` in Raw Messages.
- Plot confidence and depth over time.
- Plot `/dog/found` as a Boolean state.

Scoring-script outline

`score_oakd_detector.py` should align detections to ground-truth segments and calculate:

- `/dog/found` publication rate;
- per-segment detection fraction;
- trial-level detection success;
- false-positive frame fraction;
- detection confidence distribution;
- valid bounding-box fraction;
- median stereo depth and error relative to measured distance;
- minimum reliable distance;
- maximum reliable distance.

Define a distance as **reliably detected** when the detector reports Billie in at least 80% of sampled cycles for that segment. Report both the 80% characterization limit and the stricter system recall metric.

> **Implementation note:** implemented in `oakd/detection_scoring.py` (`align_detections_to_segments`, `compute_detection_metrics`) and `oakd/score_oakd_detector.py`, matching this list closely, with two simplifications: "detection confidence distribution" is reported as an aggregate (via the pass/fail gate on required-vs-provisional thresholds), not a saved histogram/distribution object; and `bbox_w > 0 and bbox_h > 0` is asserted for every confident detection (`valid_bbox_fraction`) — this is the regression check proving the §5.3 bounding-box scaling fix actually landed.

Pass/fail criteria

- `/dog/found` averages 4.5–5.5 Hz over active testing.
- At 1–4 m, aggregate dog recall is at least 85% under the tested indoor conditions.
- At 2.0 m, absolute median depth error is ≤0.20 m.
- Empty-scene false-positive fraction is ≤5% **provisional**.
- Positive detections have valid, nonzero bounding boxes.
- The report identifies the observed minimum and maximum reliable detection distances.

---

6. MLX90640 Thermal Camera Tests

6.1 UT-THM-01 — Thermal Frame Acquisition and Visualization

Goal

Verify electrical connectivity, I²C communication, 32 × 24 temperature-frame acquisition, 4 Hz publication, valid floating-point temperature values, and real-time visualization.

Hardware and hookup

- Raspberry Pi 5
- MLX90640 breakout board
- Four-wire I²C connection or the breakout's vendor-approved cable
- MacBook for Foxglove

With the Raspberry Pi powered off, connect:

MLX90640 function	Raspberry Pi 5 signal	Common header pin
`VCC`/`VIN`	3.3 V, subject to breakout documentation	Pin 1
`GND`	Ground	Pin 6

MLX90640 function	Raspberry Pi 5 signal	Common header pin
SDA	GPIO2 / SDA1	Pin 3
SCL	GPIO3 / SCL1	Pin 5

The exact labels and permitted supply voltage depend on the breakout board. Do not infer pin order from this table; follow the board silkscreen and vendor schematic.

Hardware preflight

Enable I²C using the host's supported configuration method, reboot if required, and save:

```
ls -l /dev/i2c-1
sudo i2cdetect -y 1
```

Expected result: device address 33 appears on bus 1.

Also run a direct Python smoke test that initializes adafruit_mlx90640, reads one 768-value frame, and prints minimum, mean, and maximum temperature. This separates hardware/library failures from ROS failures.

Launch-file outline

thermal_unit_bench.launch.py should:

1. Launch the existing production thermal_node using the production perception.yaml and mock:=false.
2. Launch thermal_colorizer.py subscribing to /thermal/image and publishing:
   - /bench/thermal/image_color as rgb8;
   - optional /bench/thermal/image_normalized as mono8;
   - a color scale or diagnostic message containing the selected min/max display temperatures.
3. Launch the common rate monitor for /thermal/image and /thermal/blob.
4. Record /thermal/image, /thermal/blob, colorized output, and diagnostics.
5. Launch Foxglove bridge.

Example intended command:

```
ros2 launch billiebot_sensor_tests thermal_unit_bench.launch.py \
  results_dir:=$HOME/billiebot_test_results/UT-THM-01 \
  duration_sec:=60 \
  record_bag:=true
```

Procedure

1. Start with the camera pointed at a typical room scene.
2. Start the launch.
3. In Foxglove, view /bench/thermal/image_color in an Image panel.
4. Move a hand through the field of view and verify a warmer region follows the hand.
5. Remove warm objects and confirm the scene returns toward ambient.
6. Leave the sensor stationary for at least 30 s to characterize temporal stability.

Analysis outline

For every /thermal/image frame, verify:

- width 32, height 24;
- encoding 32FC1;
- step 128 bytes;
- exactly 768 float values;
- finite-pixel percentage;
- frame min, mean, max, and standard deviation;
- rate and maximum frame gap;
- number and percentage of read errors from logs.

> **Implementation note:** thermal/analyze_thermal_frame.py --profile stream checks dimensions/encoding, finite-pixel fraction, and rate/gap (via thermal.dimensions/thermal.encodings in sensor_bench.yaml and common/rate_stats.py). It does not currently compute per-frame min/mean/max/std, and it does not parse console.log for MLX90640 read-error counts — open gaps.

Pass/fail criteria

- i2cdetect reports address 0x33.
- /thermal/image publishes at 3.6–4.4 Hz over 60 s.
- At least 99.9% of pixels are finite.
- Temperature values are physically plausible for the room and warm target.

- Sustained frame-read failure rate is below 1%.
- The hand or other warm target is visibly localized in Foxglove.

> **Implementation note:** rate and finite-pixel criteria are programmatically gated (required tier in `sensor_bench.yaml`). The `i2cdetect` address check and the sustained-read-failure-rate check are not — `common/preflight.py` captures `i2cdetect -y 1` output for the operator to read, but no analysis code parses or gates on it (same cross-cutting pattern noted for OAK-D in §5.1 and tracked in §10.1).

6.2 UT-THM-02 — Temperature Bias, Noise, and Contrast-to-Noise

Goal

Quantify apparent temperature bias and thermal contrast using a stable warm target and ambient background. Compute the requested signal-to-noise metric while preserving raw temperature units.

Reference-target setup

A human hand or Billie is not a precise 35 °C calibration source. Surface temperature varies and is affected by airflow, fur, emissivity, and viewing angle. Use:

- a water-filled container stabilized near 35 °C;
- a matte black tape patch on the target surface;
- a contact thermometer or thermocouple attached adjacent to the imaged patch;
- a matte background near room temperature, also measured with the reference thermometer.

Allow the target and reference probes to stabilize. Position the target so it occupies substantially more than eight thermal pixels.

Record in `manifest.yaml`:

- target reference temperature;
- background reference temperature;
- reference-instrument model and estimated uncertainty;
- camera-to-target distance;
- target ROI and background ROI;
- MLX90640 field-of-view variant if known.

Capture procedure

1. Point the sensor at the ambient background only and record 15 s.
2. Place the warm target in the scene and record 30 s without moving the camera.
3. Remove the target and record another 15 s.
4. Export representative raw frames as:
    - float32 `.npz` or `.npy` — required;
    - 32-bit float TIFF — optional;
    - colorized PNG — visual reference only.

Analysis-script outline

`analyze_thermal_frame.py` should:

1. Load the raw temperatures and ROIs.
2. For each frame, calculate target and background:
    - mean;
    - median;
    - standard deviation;
    - robust standard deviation;
    - min/max;
    - temporal drift.
3. Calculate:

```
target_bias = median(target_pixels) − reference_target_temperature
background_bias = median(background_pixels) − reference_background_temperature
delta_T = mean(target_pixels) − mean(background_pixels)
CNR = delta_T / std(background_pixels)
pooled_CNR = delta_T / sqrt(std_target^2 + std_background^2)
```

4. Compute temporal noise per pixel from the stationary sequences.
5. Generate raw heatmaps, ROI overlays, time histories, and histograms.
6. Explicitly report reference uncertainty and avoid claiming sensor calibration accuracy tighter than the reference setup supports.

> **Implementation note:** implemented in `thermal/metrics.py` (`target_background_bias`, `contrast_to_noise_ratio`, `pooled_cnr`, `temporal_noise`, `drift`) and `thermal/analyze_thermal_frame.py --profile contrast`, taking `--ref-target-c`/`--ref-background-c` and

> rectangular `--target-roi`/`--background-roi` (`x0,y0,x1,y1`) CLI arguments — same rectangle-not-polygon simplification as OAK-D's ROI (§5.2). No plots are generated (heatmaps/overlays/histories/histograms are an open gap, consistent with the export gap noted in §5.2), and there is no `--ref-uncertainty-c` argument or automatic "characterization only" labeling when reference uncertainty exceeds 1 °C — item 6 and the corresponding pass/fail carve-out below are not yet implemented; an operator must currently judge this manually from the recorded reference-instrument notes in `manifest.yaml`.

Pass/fail criteria

Initial provisional gates:

- Absolute target bias ≤2.0 °C.
- Absolute background bias ≤2.0 °C.
- CNR ≥5.
- No NaN/Inf pixels in the selected ROIs.
- No monotonic drift greater than 1.0 °C over the 30 s stationary capture.

If the reference setup uncertainty is larger than 1 °C, label the result **characterization only** and do not fail the sensor solely on the bias threshold.

6.3 DT-THM-01 — Billie Warm-Body Blob Detection

Goal

Verify the production warm-body thresholding logic on `/thermal/blob`, determine the reliable detection distance, and characterize false positives.

Algorithm under test

The current `thermal_node` performs global thresholding:

- accepted temperature: 30–40 °C;
- minimum warm area: 8 pixels;
- output: `/thermal/blob` with centroid, area, maximum temperature, mean temperature, and `is_dog_candidate`.

It is not connected-component segmentation. Multiple warm objects can merge into one global centroid. The test should document this behavior rather than treating it as a sensor failure.

Launch-file outline

`thermal_detection_bench.launch.py` should:

1. Launch the production `thermal_node`, real mode.
2. Launch the colorizer.
3. Record `/thermal/image` and `/thermal/blob`.
4. Launch a ground-truth marker that records scene labels and measured distance.
5. Launch a scorer-assist node that associates blob messages with raw image timestamps.
6. Launch Foxglove bridge.

Test conditions

Record at least 10 s per condition:

Negative:

- empty room/background for 30 s;
- human hand briefly entering the frame;
- person in frame;
- warm mug or heating vent, if safely available.

Billie present:

- 0.5 m;
- 1.0 m;
- 1.5 m;
- 2.0 m characterization;
- lying down and standing, where practical.

Keep the camera fixed and place Billie approximately centered first. Off-axis characterization can be added after the basic test passes.

Analysis-script outline

`score_thermal_blob.py` should:

- count raw frames per ground-truth segment;
- associate a blob message with a frame using timestamp tolerance;
- compute per-segment detection fraction;
- report centroid, area, mean/max temperature distributions;
- verify `area >= 8` and `30 <= mean_temp <= 40` for positive outputs;

- calculate false-positive fraction for negative segments;
- identify maximum distance meeting the configured detection-fraction threshold;
- save representative true-positive, false-positive, and false-negative frames.

> **Implementation note:** implemented in `thermal/blob_scoring.py` (`associate_blob_to_frame`, `compute_blob_metrics`) and `thermal/score_thermal_blob.py`, matching this outline closely — `thermal_node.py` itself is deliberately unmodified (per §6.3's own framing, the bench characterizes the existing global-threshold algorithm rather than replacing it). The one gap: representative TP/FP/FN frames are not saved to `exports/`, only the aggregate metrics.

Pass/fail criteria

Provisional initial gates:

- No `/thermal/blob` messages during a 30 s empty-scene baseline.
- Billie generates a blob in at least 80% of raw frames at 0.5, 1.0, and 1.5 m.
- Positive outputs have `is_dog_candidate=true`, area ≥8 px, and mean temperature in the configured 30–40 °C range.
- The report identifies the observed maximum reliable distance.
- Warm-human and warm-object false positives are reported separately; they are expected limitations of the current threshold-only algorithm and should create a software-improvement item if they are frequent.

---

7. Pi Camera 3 NoIR Tests

7.1 UT-NIR-01 — Image Acquisition and Real-Time Viewing

Goal

Verify that the Raspberry Pi detects the Camera Module 3 NoIR, Picamera2 captures frames, `/noir/image` publishes at the configured 5 Hz, and the images can be viewed in Foxglove.

Hardware and hookup

- Raspberry Pi 5
- Raspberry Pi Camera Module 3 NoIR
- **Standard-to-Mini camera cable:** 15-pin camera end to 22-pin Raspberry Pi 5 end
- MacBook for Foxglove

With the Raspberry Pi powered off:

1. Connect the 15-pin end to the camera module.
2. Connect the 22-pin end to either Raspberry Pi 5 CAM/DISP connector using the official cable orientation guidance.
3. Ensure the ribbon is square and the connector latch is closed.
4. Power on the Raspberry Pi.

Hardware/software preflight

Save output from:

```
rpicam-hello --list-cameras
rpicam-still -o $HOME/noir_preflight.jpg
python3 -c "from picamera2 import Picamera2; print(Picamera2.global_camera_info())"
```

Expected sensor identification should include the Camera Module 3/IMX708 family. Raspberry Pi OS Bookworm and later use `rpicam-*`; the host may expose equivalent libcamera commands depending on its installation.

Launch-file outline

`noir_unit_bench.launch.py` should:

1. Launch the existing production `noir_cam_node` using `perception.yaml`, `mock:=false`.
2. Launch the common topic-rate/integrity monitor for `/noir/image`.
3. Launch an optional image-quality monitor that computes brightness, clipping, repeated-frame hashes, and sharpness online.
4. Record `/noir/image` and diagnostics.
5. Launch Foxglove bridge.

Procedure

1. Point the camera at a normal indoor scene with objects at several distances.
2. Start the launch.
3. View `/noir/image` in a Foxglove Image panel.
4. Move an object and confirm that frames update rather than repeating a stale image.
5. Place a high-contrast target approximately 1 m away and confirm focus.
6. Record for 60 s.

Analysis outline

- Verify width `640`, height `480`, encoding `rgb8`, and step `1920`.
- Calculate average/median rate and maximum gap.
- Hash downsampled frames to detect frozen or repeated output.
- Compute mean luminance, black clipping fraction, white clipping fraction, and variance of Laplacian.
- Inspect logs for libcamera/Picamera2 timeouts.

> **Implementation note:** dimensions/encoding/rate/gap/repeated-frame-fraction are checked in `noir/analyze_noir_image.py --profile stream` (repeated-frame hashing via `common/image_hash.py`'s `repeated_frame_hash`, same average-hash function used across all sensors). Mean luminance and black/white clipping fraction are computed **live** instead, by `noir/image_quality_monitor.py`'s `ImageQualityMonitorNode`, published continuously to `/bench/noir/diagnostics` for Foxglove viewing during the run — they are not recomputed offline in the stream profile. Variance-of-Laplacian sharpness (`noir/metrics.py`'s `laplacian_variance`) is only computed in UT-NIR-02's quality profile, not here. Log inspection for libcamera/Picamera2 timeouts is not automated.

Pass/fail criteria

- Camera enumerates and `rpicam-still` creates a valid image.
- `/noir/image` averages at least 4.5 Hz over 60 s.
- Repeated/dropped-frame fraction is ≤1% for a scene containing motion.
- No frame gap exceeds 1.0 s.
- The image is recognizable and focused under normal indoor light.
- No sustained camera timeout or capture error occurs.

> **Implementation note:** the rate/gap/repeat-fraction criteria are programmatically gated; sustained-timeout detection is not (same cross-cutting preflight-only pattern as §5.1/§6.1, tracked in §10.1).

7.2 UT-NIR-02 — Focus, Image Quality, and Low-Light/IR Sensitivity

Goal

Quantify image quality and low-light response. This replaces the thermal-temperature test in the initial draft.

Test setup

- Camera fixed on a stable mount
- High-contrast printed chart containing black, white, and fine-edge regions
- Uniform gray card or matte wall region
- Fixed distance, recommended 1.0 m
- Measured or repeatable illumination conditions:
  - normal indoor light;
  - dim light;
  - near-darkness without IR illumination, characterization only;
  - near-darkness with an 850 nm IR illuminator, if available
- Optional lux meter; a phone lux reading may be recorded as approximate, not calibration-grade

Implementation considerations

Camera Module 3 supports autofocus. The current `noir_cam_node` does not explicitly configure autofocus or publish capture metadata. Add configurable parameters and diagnostics for:

- autofocus mode: continuous, auto, or manual;
- autofocus trigger;
- lens position;
- exposure time;
- analogue gain;
- colour gains/white balance state;
- frame duration.

Test defaults should use continuous autofocus for scene setup, then optionally lock focus for repeatable captures.

> **Implementation note — resolved:** `noir_cam_node.py` gained a pure function `build_camera_config(width, height, af_mode='', af_trigger=False, lens_position=-1.0, exposure_time_us=-1, analogue_gain=-1.0, frame_duration_limits_us=None)` where every sentinel means "don't touch" — with all args at their sentinel it returns byte-identical output to the original hardcoded `create_still_configuration()` call (proven by `test_build_camera_config_matches_current_default()`). New params `af_mode`, `af_trigger`, `lens_position`, `exposure_time_us`, `analogue_gain`, `frame_duration_us`, and `publish_metadata` all default off/sentinel. When `publish_metadata:=true`, `capture_metadata()` is published to `/bench/noir/diagnostics`. Colour gains/white balance state are not separately exposed as a settable parameter (only reported via `capture_metadata()`'s own keys when metadata publishing is on).

Capture procedure

For each lighting condition:

1. Allow auto-exposure and autofocus to settle for at least 3 s.

2. Record 10 s of `/noir/image`.
3. Export one representative lossless PNG and raw RGB `.npy` array.
4. Save camera metadata as JSON.
5. Do not move the target or camera between lighting conditions.

Analysis-script outline

`analyze_noir_image.py` should calculate:

- grayscale luminance image;
- black-patch mean and standard deviation;
- white-patch mean and standard deviation;
- gray-patch spatial noise;
- contrast-to-noise ratio:

  ```
  CNR = abs(mean_white − mean_black) / sqrt(std_white^2 + std_black^2)
  ```

- variance of Laplacian or another edge-sharpness metric;
- black and white clipping fractions;
- temporal brightness and sharpness stability;
- exposure, gain, and lens-position stability;
- optional line-spread/edge-response metrics if the chart supports them.

> **Implementation note:** implemented in `noir/metrics.py` (`contrast_to_noise_ratio`, `laplacian_variance` via a hand-rolled 3×3 kernel — no cv2/scipy, `clipping_fraction`, `temporal_stability` for the CoV checks) and `noir/analyze_noir_image.py --profile quality`, which combines multiple `--capture-dir`/`--condition` runs (one per lighting condition) into one report. Two simplifications versus this outline: (1) black/white patches are the fixed outer left/right thirds of the frame rather than an operator-configurable chart ROI (no `--target-roi`/`--background-roi` equivalent exists for NoIR yet, unlike OAK-D/thermal), and gray-patch spatial noise is not separately computed; (2) although `noir_cam_node.py` can now publish exposure/gain/lens-position metadata (`publish_metadata:=true`, see above), `analyze_noir_image.py` does not yet consume `/bench/noir/diagnostics` from the bag to compute exposure/gain/lens-position stability — that metadata is currently visible only via live Foxglove review, not folded into the quality-profile metrics. Line-spread/edge-response metrics remain unimplemented, consistent with their "optional" status in this spec.

Pass/fail criteria

Provisional gates:

Normal indoor light:

- CNR ≥10.
- Combined black/white clipping fraction ≤2%, unless the scene intentionally contains a light source.
- Autofocus reaches a stable lens position and chart edges are visibly sharp.
- Sharpness coefficient of variation over the stable interval ≤20%.

Near-darkness with 850 nm illumination:

- Target remains recognizable.
- CNR ≥5.
- No sustained capture failure.

Near-darkness without an illuminator:

- Record results for characterization only. The NoIR camera is sensitive to near-IR but does not create infrared light; failure to see a dark room without an IR source is not a camera failure.

Type 2 test status

No Type 2 detector/classifier test is defined for the Pi Camera 3 NoIR because the current BillieBot software has no node that consumes `/noir/image`. A future low-light dog detector should receive its own detector test rather than being inferred from this acquisition test.

---

8. reSpeaker XVF3800 Tests

8.1 UT-AUD-01 — USB Audio Enumeration, WAV Capture, and Signal Quality

Goal

Verify that the Raspberry Pi detects the XVF3800 as a USB audio device, records a valid 3 s PCM sample, produces audible and noncorrupt audio, and supports quantitative waveform analysis.

Hardware and hookup

- Raspberry Pi 5
- reSpeaker XVF3800 USB microphone array
- USB-C data cable
- MacBook for listening to the exported WAV

For the XVF3800 USB array, use the USB-C port on the XMOS/audio side identified by Seeed documentation, near the 3.5 mm audio jack. Do not accidentally use a microcontroller/programming-side port if the board has more than one USB connector.

Hardware preflight

Save output from:

```
lsusb
arecord -l
arecord -L
python3 -c "import sounddevice as sd; print(sd.query_devices())"
```

The ALSA card number can change between boots. Scripts should resolve the device by a stable card/device name containing `XVF3800` or `reSpeaker`, not by assuming `card 2` or `card 4`.

Direct acquisition before ROS

Before launching the production classifier, perform a direct ALSA capture. Seeed's current XVF3800 USB example uses 16 kHz, signed 16-bit little-endian PCM, and two channels. The exact exposed channels depend on installed firmware.

Example pattern:

```
arecord -D plughw:CARD=<resolved-card-name>,DEV=0 \
  -c 2 -r 16000 -f S16_LE -d 3 ambient.wav
```

The test script should discover and print the resolved ALSA device rather than requiring manual card-number edits.

Record at least:

1. `ambient.wav` — 3 s quiet-room background;
2. `speech.wav` — 3 s of a human speaking at approximately 1 m;
3. `impulse.wav` — 3 s containing one or two hand claps, at a safe level.

ROS test-node/launch outline

The production `audio_classifier` does not publish raw audio. Implement `audio_capture_node.py` for bench use. It should:

- open the resolved XVF3800 input;
- use a continuous callback stream rather than repeated blocking `sd.rec()` calls;
- publish optional raw audio blocks using a documented standard ROS audio message if that dependency is adopted;
- continuously report sample rate, channel count, overflow count, RMS, peak, and clipping diagnostics;
- save a requested-duration WAV on service or launch command;
- use a bounded queue so disk writes do not block the audio callback.

`audio_capture_bench.launch.py` should launch the capture node, diagnostics, optional recorder, and Foxglove bridge. Foxglove waveform plotting is optional; the WAV and offline plots are the primary evidence.

> **Implementation note:** shipped as `audio/audio_capture_node.py` (class `AudioCaptureNode`), reusing the production package's `billiebot_audio.audio_device.resolve_input_device` for stable device selection (see §8.2's DoA/device-resolution notes) — a genuinely continuous `sounddevice.InputStream` feeding a bounded `queue.Queue`, never a blocking `sd.rec()`. Its `/bench/audio/diagnostics` topic reports device name, sample rate, channel count, overflow count, and blocks-written continuously, but does **not** report live RMS/peak/clipping — those are computed offline by `analyze_audio.py` from the saved WAV, not streamed live. No raw-audio ROS topic is published (the doc's own item already frames this as optional "if that dependency is adopted" — it was not adopted).

Export to MacBook

Copy the WAV files to the MacBook, for example:

```
scp <pi-user>@<pi-ip>:~/billiebot_test_results/UT-AUD-01/exports/*.wav .
```

Listen using QuickTime Player, VLC, Audacity, or another normal audio player. Record qualitative notes for clarity, hum, dropouts, channel imbalance, and distortion.

Analysis-script outline

`analyze_audio.py` should load the WAV with `scipy.io.wavfile`, `soundfile`, or Python's `wave` module and calculate per channel:

- sample rate;
- sample count and duration;
- mean/DC offset;
- standard deviation;
- RMS and RMS dBFS;
- peak amplitude and peak dBFS;
- clipping fraction;
- zero/silent-sample fraction;
- channel correlation and channel RMS imbalance;
- waveform and spectrogram;
- optional mains-hum energy around 50/60 Hz and harmonics.

For speech/impulse recordings, calculate signal-to-background ratio using `ambient.wav` as the noise reference.

> **Implementation note:** implemented in `audio/metrics.py` (`rms_dbfs`, `peak_dbfs`, `clipping_fraction`, `dc_offset`, `channel_correlation`, `mains_hum_energy` via `numpy.fft`, no scipy) and `audio/analyze_audio.py`, using Python's `wave` module. Not yet implemented: zero/silent-sample fraction, channel RMS imbalance (only correlation is computed), waveform/spectrogram plots (`analyze_audio.py` produces no plots at all today), and the ambient-referenced signal-to-background-ratio calculation for speech/impulse recordings — each WAV is scored independently. These are open gaps for a more complete UT-AUD-01 analysis pass.

Pass/fail criteria

- The device enumerates in both USB and ALSA inventories.
- Each WAV is readable on Linux and the MacBook.
- Duration is 3.00 ±0.05 s.
- Sample rate is 16,000 Hz.
- Channel count matches the selected XVF3800 firmware/profile and is documented.
- Audio is nonzero and clearly audible for speech/impulse recordings.
- Clipping fraction is <0.1%.
- Absolute normalized DC offset is <0.01 full scale **provisional**.
- No ALSA overflow/underflow is reported.

The standard deviation requested in the initial draft is included, but it is not sufficient by itself to judge audio quality.

8.2 DT-AUD-01 — Bark, Speech, and Other-Sound Classification

Goal

Verify the production YAMNet classifier, measure actual processing cadence, quantify bark detection, and characterize how non-dog audio is represented by the current `AudioEvent` interface.

Production-software preconditions

- A compatible `yamnet.tflite` model is staged on the Raspberry Pi.
- `yamnet_class_map.csv` is in the same directory as the model.
- `audio.yaml` contains the actual model path; it is currently empty in the repository and must be configured before real testing.
- The `sounddevice` input device is resolved to the XVF3800 explicitly rather than relying on the system default.
- `tflite_runtime` or TensorFlow Lite is installed.

Required implementation correction for 2 Hz processing

The current node creates a 0.5 s timer but performs a blocking 0.975 s recording inside each callback. In a normal single-threaded ROS executor this cannot sustain a true 2 Hz processing cadence. Refactor to:

1. open one continuous `sounddevice.InputStream`;
2. write incoming samples into a ring buffer;
3. run inference on a 0.975 s window every 0.5 s;
4. keep audio capture outside the inference callback;
5. publish `/bench/audio_classifier/status` every processing cycle with:
    - cycle timestamp;
    - inference duration;
    - audio-window start/end timestamps;
    - energy gate result;
    - top label and score, even when no `/audio/events` message is emitted;
    - buffer overrun count.

`/audio/events` should remain event-driven.

> **Implementation note — resolved (with one field gap):** the continuous-capture refactor is implemented in `audio_classifier.py` almost exactly as prescribed — a new `AudioRingBuffer` class (new file `billiebot_audio/audio_ring_buffer.py`, a `threading.Lock`-guarded circular `np.float32` buffer) is filled by a `sounddevice.InputStream` callback on its own PortAudio thread; the existing 0.5 s ROS timer now does a near-instant `read_last(...)` followed by synchronous TFLite inference (bounded, tens of ms — safe to block on, unlike the old 0.975 s capture). New param `continuous_capture` defaults **`true`** (a deliberate exception to "defaults off" — the old blocking-capture path is a confirmed non-functional bug, not a working baseline worth preserving as the default; `continuous_capture:=false` reproduces it verbatim as `_real_classify_legacy_blocking()`, an explicit rollback). `/bench/audio_classifier/status` (`publish_status`, default `true`) is published every cycle as a `diagnostic_msgs/DiagnosticArray` with `cycle_timestamp_monotonic_s`, `inference_duration_s`, `energy_db`, `passed_energy_gate`, `top_label`, `top_score`, and `buffer_overrun_count`. The one gap versus this list: explicit audio-window start/end timestamps are not separately reported (only the cycle timestamp, which approximates the window's end).

Current speech behavior

`AudioEvent.msg` has BARK, WHINE, HOWL, LOUD_NOISE, and SILENCE; it has no SPEECH enum. The current code maps every confident non-dog YAMNet class to `LOUD_NOISE` while retaining the original class in `yamnet_label`. Therefore, under the current interface the expected result for human speech is:

- `yamnet_label`: a speech-related YAMNet label;
- `event_type`: `LOUD_NOISE`.

If BillieBot requires an explicit human-speech class, that is a separate interface and cognition change.

Launch-file outline

`audio_classifier_bench.launch.py` should:

1. launch the production `audio_classifier` in real mode with the configured model and XVF3800 device;
2. launch classifier diagnostics/status;
3. record `/audio/events`, `/bench/audio_classifier/status`, and test ground truth;
4. optionally save the raw input WAV for every trial;
5. launch a ground-truth marker service or consume a segment CSV;
6. launch Foxglove bridge for status plots.

> **Implementation note:** items 1, 3, 5, and 6 are implemented as described (production `audio_classifier` via `replicate_production_node`, rosbag2 recording of both topics, the same stdin-driven `ground_truth_marker_node` used for DT-OAK-01/DT-THM-01, Foxglove bridge). Item 4 — saving a per-trial raw WAV clip — is not implemented; the continuous bag recording is the only audio evidence retained.

Test dataset and procedure

Use prerecorded clips first for repeatability. Play them from an external speaker positioned at measured distances from the array. Do not play through the XVF3800's own speaker output during microphone testing unless echo-cancellation behavior is specifically under test.

Recommended classes:

- Billie bark clips: at least 20 events if available;
- other dog vocalizations: whine/howl if available;
- human speech: at least 20 phrases/events;
- non-speech impulse/noise: at least 20 claps, knocks, or household sounds;
- silence/ambient: at least 60 s.

Recommended distances:

- 0.5 m;
- 1.0 m;
- 2.0 m;
- 3.0 m characterization.

After prerecorded testing, perform a short live-Billie confirmation using naturally occurring barks.

Scoring-script outline

`score_audio_classifier.py` should:

- align status and event messages to labeled audio intervals;
- measure processing-cycle frequency independently of event publication;
- calculate cycle jitter, inference latency, and end-to-end event latency;
- calculate a confusion matrix for BARK, WHINE/HOWL if available, LOUD_NOISE, and no-event;
- calculate bark precision, recall, and F1;
- calculate false bark classifications on speech and other noises;
- report YAMNet top-label distributions for human speech;
- calculate confidence and energy distributions by class and distance;
- preserve representative false positives and false negatives with their WAV clips.

> **Implementation note:** implemented in `audio/classifier_scoring.py` (`precision_recall_f1`, `latency_stats`, `label_distribution`; a pure `confusion_matrix` function also exists and is unit-tested) and `audio/score_audio_classifier.py --profile classification`. Status-cycle frequency is measured independently of `/audio/events` via `common/rate_stats.py` on `/bench/audio_classifier/status` (this is the regression check proving the continuous-capture refactor actually reaches ~2 Hz). Bark precision/recall/F1, false-bark rate on non-bark segments, and event latency

> are all computed and gated. Gaps versus this outline: the full multi-class confusion matrix (BARK/WHINE-HOWL/LOUD_NOISE/no-event) is not wired into the scoring CLI's output even though the pure function backing it exists; confidence/energy are not broken down by class-and-distance (only aggregated); and representative false-positive/false-negative WAV clips are not preserved separately from the full session bag.

Pass/fail criteria

- Processing status averages 1.8–2.2 Hz with no audio-buffer overflows.
- Aggregate bark recall is ≥80% for the defined acceptance distances and playback level.
- False BARK classification on human-speech and non-dog-noise trials is ≤10% **provisional**.
- Event latency from the end of the relevant audio evidence to publication is ≤1.5 s **provisional**.
- Human speech produces a speech-related `yamnet_label`; under the current interface `event_type=LOUD_NOISE` is expected.
- Silence below the energy threshold produces no `/audio/events` output, while the status topic continues at the processing rate.

8.3 DT-AUD-02 — Direction of Arrival, Optional but Recommended

Goal

Verify XVF3800 direction-of-arrival output independently from audio classification.

Known integration risk

The production code currently searches USB VID/PID `0x2886:0x0018`, associated with an earlier ReSpeaker family. Current Seeed XVF3800 ROS 2 documentation identifies the XVF3800 with product ID `0x001A`. Codex should not simply replace one constant without validation; it should enumerate the connected device and use the official XVF3800 host-control API or supported ROS 2 driver.

> **Implementation note — resolved:** `audio_classifier.py`'s `_get_doa()` now enumerates all connected USB devices (`usb.core.find(find_all=True)`) and resolves the DoA-capable device via a new pure function `resolve_doa_device(candidates, product_substring)`: it prefers a device whose product-name string matches `doa_usb_product_substring` (new param, default `'XVF3800'`), and only falls back to the legacy `0x2886:0x0018` VID:PID if no name match is found — so hardware that happened to enumerate under the old ID never regresses. This did not replace one hardcoded constant with another, per the concern raised here.

Procedure

1. Mount the array in a fixed orientation and mark its defined 0° forward direction.
2. Place a speaker at 1.0 m at bearings 0°, 45°, 90°, 135°, 180°, 225°, 270°, and 315°.
3. Play the same speech or broadband sound clip for 10 s at each bearing.
4. Record raw DoA estimates and timestamps.
5. Repeat at 2.0 m if the room permits.

Analysis

Use circular statistics:

- circular mean DoA;
- circular standard deviation;
- circular absolute error relative to ground truth;
- percentage within ±15°;
- dropout or default-zero fraction.

> **Implementation note:** implemented in `common/circular_stats.py` (`circular_abs_error_deg`, correctly handling the 0°/360° wraparound — e.g. true=359°, measured=1° ⇒ error=2°, not 358°; also `circular_mean_deg`/`circular_std_deg` as general-purpose functions) and `audio/score_audio_classifier.py --profile doa`, which pairs each `/audio/events.doa_deg` sample against the operator's marked bearing (`orientation=<deg>` in the stdin ground-truth grammar). Mean absolute circular error and the within-±15° fraction are computed and gated. Two simplifications: the circular mean/std of the raw *measured* DoA values themselves is not reported (only the error relative to ground truth); and "dropout or default-zero fraction" is reported as a boolean (`doa_all_zero`, true only if every sample was exactly 0°) rather than a fraction of zero/dropout samples.

Pass/fail criteria

- Mean absolute circular error ≤15° at the four cardinal bearings.
- At least 80% of samples are within ±15° under the controlled test **provisional**.
- DoA does not remain fixed at 0° for all source positions.
- The USB device and control interface used are recorded in the report.

A classifier pass does not depend on this optional test; DoA is a separate hardware/API capability.

---

9. Recommended Test Execution Order

Run only one sensor at a time.

1. **OAK-D Lite**
    1. USB/DepthAI preflight
    2. UT-OAK-01 stream test

    3. UT-OAK-02 depth test
    4. DT-OAK-01 dog detection
2. **MLX90640**
    1. I²C preflight
    2. UT-THM-01 stream test
    3. UT-THM-02 thermal contrast test
    4. DT-THM-01 Billie blob test
3. **Pi Camera 3 NoIR**
    1. CSI/rpicam preflight
    2. UT-NIR-01 stream test
    3. UT-NIR-02 image-quality and low-light test
4. **reSpeaker XVF3800**
    1. USB/ALSA preflight
    2. UT-AUD-01 direct audio capture
    3. Configure YAMNet assets and device selection
    4. DT-AUD-01 classifier test
    5. DT-AUD-02 DoA test, optional

Do not proceed to a detector/classifier test until the associated acquisition test passes. This prevents algorithm debugging from being confused with wiring, USB, I²C, CSI, or driver failures.

---

10. Codex/Claude Code Implementation Requirements

The implementation prompt generated from this plan should require the following engineering behaviors.

10.1 General requirements

- Use production message types and production algorithm nodes for Type 2 tests.
- Keep test-only topics under `/bench/...`.
- Do not require the robot chassis, map, TF tree, lidar, motors, or navigation stack.
- Fail loudly and exit nonzero when a required device, model, topic, or data file is missing.
- Resolve USB and ALSA devices by stable identity, not enumeration order.
- Record the repository commit and all effective ROS parameters.
- Use rosbag2 for authoritative time-series capture.
- Export human-readable artifacts in addition to rosbag2.
- Make all acceptance limits configurable.
- Produce `metrics.json` and `report.md` automatically.
- Add unit tests using synthetic arrays/WAV files for every analysis script.
- Avoid changing production defaults solely for bench visualization; expose optional parameters that default off.

> **Implementation notes — cross-cutting gaps found across every test in this document, tracked here once rather than repeated per section:**
>
> 1. **Preflight results do not gate pass/fail anywhere.** `common/preflight.py` runs and saves the prescribed hardware/software preflight commands (`lsusb`, `i2cdetect -y 1`, `rpicam-hello --list-cameras`, `arecord -l`/`-L`, the DepthAI/picamera2/sounddevice smoke-test one-liners) to `exports/preflight.json` and `console.log` for every Type 1 test, but no `analyze_*`/`score_*` CLI parses that output to programmatically enforce the "device enumerates," "i2cdetect reports 0x33," or "no sustained read/capture error" pass/fail criteria stated throughout §5–§8. These remain manual/qualitative checks today.
> 2. **`manifest.yaml`'s `parameters:` field is always `{}`.** Effective ROS parameters are never captured, despite this being an explicit, repeated requirement (see the §3.3 note above and §10.4 below).
> 3. **No analysis CLI exports representative frames.** Despite §1.1 item 4 and the individual test procedures repeatedly requiring a representative RGB PNG / depth `.npz` / thermal `.npz` / NoIR PNG+`.npy` to be written to `exports/`, none of `analyze_oakd_depth.py`, `analyze_thermal_frame.py`, or `analyze_noir_image.py` currently does this — the rosbag2 recording is the only artifact produced beyond WAV files (audio) and the ground-truth CSV. An operator can extract representative frames from the bag manually with `common/bag_reader.py`'s `BagReader` in the meantime.
>
> Resolving all three is a reasonable next PR; none of them block a first hardware bench run, since the bag remains the authoritative record either way.

10.2 Data-unit requirements

- OAK depth encoding must state whether values are millimetres or metres.
- Point clouds must state frame ID and units.
- Thermal arrays must remain float degrees Celsius in the raw record.
- Image encodings must match byte order and channel order.
- Audio should use documented PCM format and normalize carefully during analysis.
- Distances and angles in operator-entered ground truth must include units.

10.3 Timing requirements

- Use message header time when valid and receipt time as a secondary measurement.
- Report both sensor publication rate and detector output/event rate.
- For audio, report capture continuity, analysis-window hop rate, inference duration, and event latency separately.

- Do not infer a 2 Hz classifier rate from sparse `/audio/events` messages.

10.4 Reproducibility requirements

- Save exact commands and parameters.
- Save the selected ROIs and ground-truth segments.
- Save the Foxglove layout or provide a layout file.
- Use deterministic random seeds for analysis tests.
- Do not overwrite prior results; use timestamped directories.

> **Implementation note:** commands are saved (`manifest.yaml`'s `launch_command`); parameters are not yet (§10.1 gap #2 above). Ground-truth segments are saved (`exports/ground_truth_segments.csv`); ROIs are only saved if the operator manually writes the JSON/CLI-arg file themselves (§5.2/§6.2/§7.2 notes) — no automatic ROI persistence exists yet. The Foxglove layout is shipped (`foxglove/billiebot_sensor_bench.json`, §3.4 note). Deterministic seeds are used throughout `test/fixtures/*` (`numpy.random.default_rng(seed)` with explicit integer seeds). `BenchResultDir.create()` raises if `results_dir` already exists and is non-empty, and `run_sensor_test`'s default results directory is UTC-timestamped — prior results are never silently overwritten.

11. Known Repository Findings Relevant to These Tests

The test suite should explicitly confirm or expose the following current-code observations:

Area	Current observation	Test implication	Status (as of `19f8513`)
OAK-D preview	Production detector publishes detections and found flag, not images	Add optional preview/diagnostic outputs for bench use	**Resolved** — `publish_preview`/`publish_depth_preview`/`publish_diagnos` params, default off (§5.3)
OAK-D bounding box	Normalized DepthAI coordinates are cast directly to integers	Detection test should fail nonpositive bounding boxes until scaling is corrected	**Resolved** — `scale_bbox()`, unconditional, Appendix B-11 (§5.3)
OAK-D model	`perception.yaml` supplies `/home/sean/billiebot/models/yolov8n_416.blob`; node now fails loud if missing	Preflight should verify this exact effective path	**Open** — fail-loud behavior predates this package (GAP-11); preflig does not independently re-verify the effective `model_path`
Thermal algorithm	One node publishes raw image and performs global warm-pixel thresholding	Blob test must distinguish algorithm limitations from sensor health	**Resolved (by design)** — `thermal_node.py` deliberately unmodifi DT-THM-01 characterizes the existing behavior (§6.3, §5b of the implementation record)
Thermal display	Raw topic is `32FC1`	Add a colorizer for Foxglove but retain raw float data	**Resolved** — `thermal/thermal_colorizer.py`, raw topic untouc
NoIR detector	No production node consumes `/noir/image`	Do not invent a Type 2 test yet	**Unchanged, as intended** — still no Type 2 NoIR test
NoIR focus	Current node does not explicitly set AF controls or publish metadata	Image-quality test should add AF/exposure diagnostics	**Partially resolved** — optional AF/exposure/gain/metadata params implemented, default off (§7.2); the quality-profile analysis CLI doe yet consume the published metadata
Audio model	`audio.yaml` currently has an empty `model_path`	Real classifier test cannot start until assets are staged/configured	**Partially resolved** — `model_path`/class-map validation is now fail- (`sys.exit(1)`) instead of a soft warning; `audio.yaml`'s `model_p` still empty by design and must be staged by the operator
Audio cadence	0.975 s blocking record in a 0.5 s timer	Refactor to continuous capture/ring buffer before enforcing 2 Hz	**Resolved** — `AudioRingBuffer` + continuous `InputStream`, `continuous_capture` default `true` (§8.2)
Audio speech class	No SPEECH enum; non-dog classes map to LOUD_NOISE	Score raw `yamnet_label` and document interface behavior	**Resolved** — unchanged interface, documented and scored as-is in `score_audio_classifier.py` (§8.2)

Area	Current observation	Test implication	Status (as of `19f8513`)
XVF3800 DoA	Production code uses older ReSpeaker USB ID	Enumerate hardware and use XVF3800-supported API/driver	**Resolved** — `resolve_doa_device()` enumeration with legacy-ID fallback (§8.3)

Formally closing the GAP-tracker entries this table references (Appendix B-11, B-14) — updating `DISCREPANCY_RESOLUTION_PLAN.md` §2/§4, `BRINGUP_LADDER_ANALYSIS.md` Appendix B, and `MBSE_SYSTEM_DECOMPOSITION.md` §5.4, then closing the corresponding GitHub issues — is a separate follow-up PR per the repo's own three-location-plus-issue update rule, not done as part of this package.

---

12. Completion Criteria

The sensor bench campaign is complete when:

1. Every Type 1 test has a timestamped result directory, rosbag2 capture, exported sample data, metrics file, and signed-off report.
2. OAK-D Lite passes streaming and 2 m depth-accuracy tests.
3. MLX90640 passes streaming and controlled target/background characterization.
4. Pi Camera 3 NoIR passes streaming and normal-light image-quality testing; IR-assisted low-light results are recorded if an illuminator is available.
5. XVF3800 records valid, audible WAV files with acceptable signal integrity.
6. OAK-D, thermal, and audio Type 2 tests produce a scored result, including false positives and false negatives rather than only visual confirmation.
7. Every observed failure is classified as one of:
    - hardware/wiring;
    - host driver/library;
    - ROS transport/topic;
    - test harness;
    - algorithm/model;
    - configuration;
    - environmental limitation.
8. Any failed acceptance item is entered as a repository issue with the test result directory and key evidence attached.

---

13. External Technical References

- Luxonis OAK-D Lite documentation: https://docs.luxonis.com/hardware/products/OAK-D%20Lite
- Raspberry Pi camera hardware documentation: https://www.raspberrypi.com/documentation/accessories/camera.html
- Raspberry Pi camera software documentation: https://www.raspberrypi.com/documentation/computers/camera_software.html
- Melexis MLX90640 product page: https://www.melexis.com/en/product/mlx90640/far-infrared-thermal-sensor-array
- Adafruit MLX90640 guide: https://learn.adafruit.com/adafruit-mlx90640-ir-thermal-camera
- Seeed Studio XVF3800 introduction: https://wiki.seeedstudio.com/respeaker_xvf3800_introduction/
- Seeed Studio XVF3800 ROS 2 guide: https://wiki.seeedstudio.com/respeaker_xvf3800_ros2/

End of test plan.